

Program-5

Experiment: Locally Weighted Regression (LWR)

Aim

To implement the Non-Parametric Locally Weighted Regression (LWR) algorithm to fit the given data points.

Theory

Locally Weighted Regression (LWR) is a non-parametric regression technique used to model complex relationships between variables. Unlike traditional regression methods, LWR does not assume a fixed model. Instead, it builds a separate regression model for each query point using nearby data.

In LWR, points closer to the query point are given higher importance, while distant points are given lower importance. This is achieved using a weighting function.

The weight of each data point is calculated using the Gaussian kernel:

$$w_i = e^{-\frac{(x_i - x_0)^2}{2\tau^2}}$$

where

(w_i) = weight of the data point

(x_i) = input data point

(x_0) = query point

(τ) = bandwidth parameter controlling smoothness

A weighted linear regression is then performed using these weights to predict the output value. The prediction is given by:

$$\hat{y} = X (X^T W X)^{-1} X^T W Y$$

where

(X) = input feature matrix

(Y) = output vector
 (W) = diagonal weight matrix

The parameter (τ) plays an important role:

- Small (τ) : model becomes complex and may overfit
- Large (τ) : model becomes smoother and may underfit

Thus, LWR is useful for fitting non-linear data patterns.

Result

The Locally Weighted Regression algorithm was successfully implemented and applied to the given dataset. The model produced a smooth curve that fits the data points effectively.

It was observed that:

- The model adapts well to local variations in the data
- Smaller values of (τ) lead to overfitting
- Larger values of (τ) lead to underfitting

Hence, LWR provides a flexible approach for modeling non-linear relationships between variables.

Theory

Locally Weighted Regression (LWR), also known as LOESS or LOWESS, is a non-parametric regression algorithm. It fits a model locally around each query point instead of assuming a global function.

Nearby points are given higher weights, while distant points have less influence.

The weight is calculated using a Gaussian kernel:

$$w(i) = \exp(-(x(i) - x)^2 / (2 \cdot \tau^2))$$

Small τ → more flexible curve

Large τ → smoother curve

Procedure

1. Import required libraries.
2. Create or load dataset.
3. Define Gaussian kernel.
4. Implement LWR algorithm.
5. Predict outputs.
6. Plot graph.
7. Analyze results.

Program

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("BostonHousing.csv")

# Take only one feature (important)
X = df['rm'].values
Y = df['medv'].values

# Sort data (for smooth graph)
sort_idx = np.argsort(X)
X = X[sort_idx]
Y = Y[sort_idx]

tau = 0.5
Y_pred = []

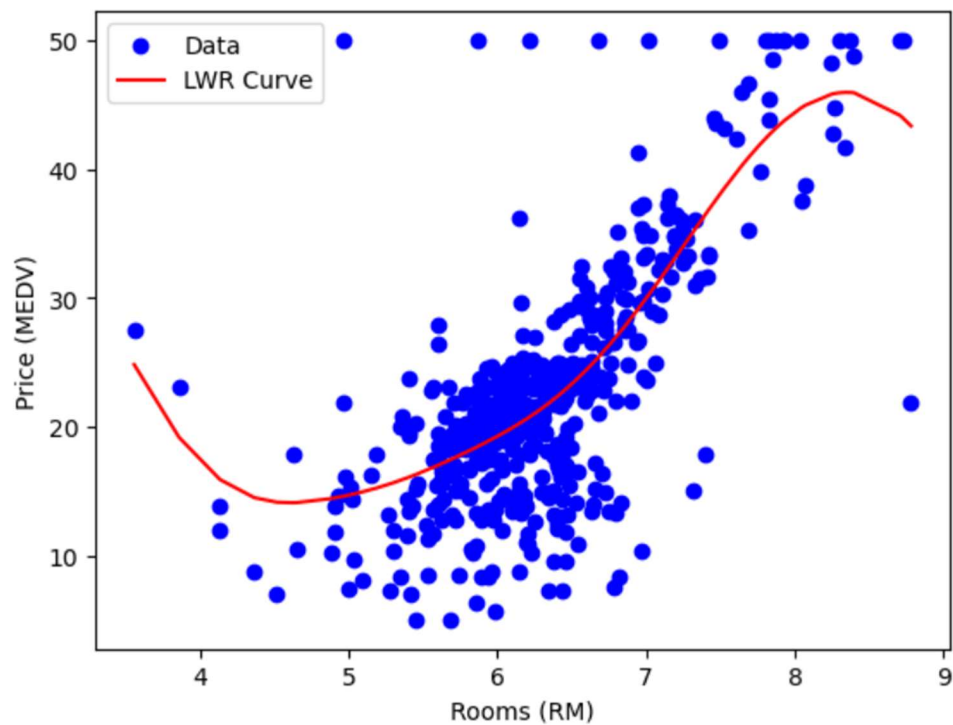
# LWR
for x0 in X:
    w = np.exp(-(X - x0)**2 / (2 * tau**2))

    X1 = np.c_[np.ones(len(X)), X]
    W = np.diag(w)
```

```
theta = np.linalg.pinv(X1.T @ W @ X1) @ (X1.T @ W @ Y)

y0 = theta[0] + theta[1] * x0
Y_pred.append(y0)

# Graph
plt.scatter(X, Y, color='blue', label='Data')
plt.plot(X, Y_pred, color='red', label='LWR Curve')
plt.xlabel("Rooms (RM)")
plt.ylabel("Price (MEDV)")
plt.legend()
plt.show()
```



Result

The LWR algorithm was successfully implemented. The regression curve fits the data well and adapts locally based on the bandwidth parameter.